A Hyper-Heuristic Framework for Adaptive Task Scheduling in Heterogeneous Cloud Environments Project Report

MSc Research Project
Cloud Computing

Olumide Akinkunmi
Student ID: 23351411

School of Computing
National College of Ireland

Supervisor: Yasantha Samarawickrama

November 2026

Abstract

In the rising era of cloud-based applications, physical hardware management is delegated to data center companies. These cloud environments are supposed to work with fluctuating requests from various clients, while handling the requests as quickly as possible. During this, they are also supposed to maximize the resources available. Traditional scheduling slgortihms are not sophisticated enough to handle dynamic workloads. This report presents a Multi-Armed Bandit hyper-heuristic framework for task scheduling. It integrates a low-level heuristic pool of various traditional scheduling algorithms, and tests them with a Java CloudSim backend.

Four selection algorithms are compared against each other: ϵ-Greedy, Standard UCB, Discounted UCB, and Scaled Thompson Sampling. The scheduler implements a reward function that optimizes makespan, execution cost and resource utilization. We use 2 varying worktraces: HPC2N and NASA datasets. After running the simulations, we get to see that the Discounted UCB selection mechanism provided the best results for highly varying workloads.

1 Introduction

1.1 Background and Motivation

As the demand for compute becomes larger than ever, data centers become a very critical part of modern digital infrastructure, because it gives organizations access to a scale of computational power which, in most cases, cannot be managed on-premises. To manage these physical machines means to efficiently assign arriving requests or tasks to Virtual Machines, while in accordance with SLA violations and maximizing hardware utilization. The problem now comes down to finding the best mode of allocation in a heterogeneous cloud environment.

With the emergence of AI/ML workloads, it becomes more important than ever to find a way to efficiently and consistently assign workloads to the right Virtual Machine Instance. These days, data centers are beginning to face heavier batch jobs that are also computationally heavy. Modern data center managers are also responsible for balancing trade-offs that aim to reduce makespan and financial costs, while also trying to maintain high hardware utilization Jeba et al. (2019).

1.2 Problem Statement

Traditional scheduling algorithms such as Heterogeneous Earliest Finish Time, Min-Min and Max-Min have fixed rules that they follow, in terms of task metadata and resource availability; their strengths become their shortcomings in the context of a heterogeneous cloud environment. Simulated Annealing, a meta-heuristic algorithm uses finer-grained processes to schedule tasks Kumar et al. (2019). The major issue with this approach is the fact that it is very computationally expensive, therefore not making them an ideal candidate for batch workloads.

As mentioned above, static or traditional scheduling algorithms cannot simply adapt to constantly changing workloads, and when this happens, resources are underutilized, SLAs are violated, and operational costs balloon

1.3 Proposed Solution

This project presents a dynamic Multi-Armed Bandit (MAB) hyper-heuristic framework for task scheduling heterogeneous cloud environments. Instead of directly searching the task-to-VM space, it dynamically selects the low-level heuristic that provides the most optimal result from a pool of scheduling algorithms.

The framework consists of a Python decision plane and a Java CloudSim plus simulation application which communicate via Inter Process Communication(IPC). The framework uses a multi-objective reward function that tanh squashes multiple execution metrics into a single, readable score between 0 and 1.

1.4 Key Contributions

The main contributions of this dissertation are:

- **Decoupled Hyper-Heuristic Architecture:** We build a lightweight, streaming Python-CloudSim Plus framework that selects low-level heuristics dynamically without interrupting execution streams.
- **Non-Stationary Adaptation under Failure:** In this project, we show that Discounted UCB performs better than other bandit algorithms when system failures occur.
- **Multi-Objective Reward Model:** Instead of using the output got from the simulation verbatim, we combine all the individual features into a singular reward.

1.5 Report Structure

The rest of this report is organized as follows: Section 2 focuses on literature review of previous works. Section 3 expands on the methodology and reasoning behind decisions made while working on this project. Section 4 presents the system design and architecture. Section 5 explains how the project was implemented. Section 6 presents experimental results and statistical evaluations. Finally, Section 7 concludes the report and gives recommendations for future work.

2 Related Work

The landscape of task scheduling in data centers keeps evolving. It has moved from traditional prioritization towards online learning frameworks. Traditional scheduling focused on optimizing just one objective such makespan, using algorithms like Heterogeneous Earliest Finish Time and Predictive Earliest Finish Time (Hosseini Shirvani and Noorian Talouki; 2021). These traditional algorithms are heavily reliant on the fact that the physical hardware is statically available; it does not factor in downtime into its scheduling process. This makes it suffer serious performance degradation when it experiences hardware faults. To help with these limitations, newer algorithms like Hybrid Swarm Optimizers were developed (Alkaam et al.; 2025).

The latest advancements in the space seem to have moved towards utilizing Deep Reinforcement Learning and Multi-Armed Bandit Algorithms (De Carvalho and Sichman; 2025; Khan et al.; 2025). Using the Deep Reinforcement Learning methodology provides

stellar results in stationary environments, but have very high training overhead (Choppara and Lokesh; 2025; Sravan and Shaik; 2026). Multi-Armed Bandit gives the option of using hyper-heuristics for lightweight online learning (Singh et al.; 2024)

There are still three major gaps in the current literature:

1. **Historical Bias in Non-Stationary Environments:** Standard MAB algorithms such as UCB and standard Thompson Sampling keep a very long performance history window, which makes it difficult for them to quickly react to unexpected capacity drops, or other hardware failures.

2. **Lack of Lightweight Contextual Workload Fingerprinting:** Existing contextual or content-aware schedulers often rely on heavy offline classifiers or neural feature extractors (Adil et al.; 2022).

3. **Unbalanced Multi-Objective Reward Formulations:** Although carbon intensity is now a major factor to be considered while scheduling (Arora et al.; 2023), existing multi-objective reward functions often get distorted across multiple batches.

The remainder of this section critically reviews prior contributions across these foundational domains.

2.1 Foundations of Non-Stationary Workload Allocation

In heterogeneous cloud environments where tasks and requests don't all arrive in a sequential or predictable order, allocating resources becomes an NP-hard problem.

Mathew et al. (2014) categorised different scheduling algorithms around performance measures such as makespan, load balancing and throughput. Saurav and Benedict (2021) takes this further by looking into energy-aware task scheduling in large, heterogeneous systems, and bargaining and exchange between execution speed and power consumption. All literature referenced in this section come to the same conclusion that traditional scheduling algorithms always underperform in heterogeneous environments.

2.2 Deterministic Task Prioritization and Heuristic Approximations

List-based algorithms like Heterogeneous Earliest Finish Time (HEFT) and Predictive Earliest Finish Time (PEFT) work in two stages: firstly, they rank the tasks, then they assign them to processors.

Hosseini Shirvani and Noorian Talouki (2021) uses this approach as a foundation to propose HH-LiSch: a hybrid list scheduler that combine two ranking strategies RankI and RankO. It was tested on benchmarks like Montage and Cybershake, and it saw a reduction in average makespan by 16.71%, compared to HEFT and PEFT.

The major limitation is still the fact that classical list schedulers use fixed rules and limited workload information, and they do not learn from changing task characteristics.

2.3 Content and Workload Feature Engineering for Scheduling

Instead of treating each task as a singular unit of computation, using feature-informed schedulers use properties of the workload itself to examine task characteristics and determine resource allocation.

Adil et al. (2022) proposed CA-MLBS, a solution that combines both Support Vector Machine classification and Particle Swarm Optimization. The algorithm works by classifying incoming tasks by content type and maps them onto the most ideal machine available. Experiments showed that this method reduced makespan by up to 29% and increased throughput by 44% against feature-agnostic algorithms.

A major deterrent with this approach is that the classifier is trained offline, which means that once it is deployed, it does not adapt to changes in hardware or workload conditions.

2.4 Complex Search Space Exploration in Hybrid Meta-Heuristics

Kumar and Rai (2021) applied Ant Colony Optimization (ACO) in CloudSim and reported lower average task execution times and cloudlet failure rates than Round Robin and Minimum Completion Time (MCT) policies.

Recent studies have gone further by combining several meta-heuristics, like Alkaam et al. (2025), who proposed HGHHC: which combines Henry Gas Solubility Optimization, Harris Hawks Optimization, and Comprehensive Opposition-Based Learning (COBL). Across HPC2N, NASA, and synthetic workload traces, HGHHC reduced makespan by 34.30%, 72.95%, and 28.67% respectively, relative to the reported baseline swarm methods. Bal et al. (2022) also proposed RATS-HM, which combines an improved cat swarm optimization scheduler (ICS-TS), a group optimization deep neural network (GO-DNN) and NSUPREME encryption for resource allocation and security.

The downside of this is that it is very computationally expensive, as these methods most likely require multiple population-based searched to properly assign tasks to resources. Having to repeatedly run this process for constantly changing workloads is expensive and may as well be too slow.

2.5 Adaptive Online Decision-Making and Regret Minimization

Multi-Armed Bandit models continuously learns from previous actions, rather than searching from scratch. The trade-off is between exploration and exploitation: where the algorithm tries lesser known actions, or it continues with actions that have previously performed well.

Singh et al. (2024) proposed a MAB model for task scheduling that derives logarithmic regret bounds, as well as proposes latency and utilization improvements. Tran-Dang and Kim (2024) merged Thompson Sampling with matching theory for a decentralised taks offloading through MAB-MATCH. Chigu et al. (2024) developed an Upper Confidence Bound scheduler for serverless FaaS systems. Multi-Armed Bandits can also be used for federated learning under mobility constraints, as well as satellite beamforming (Jun et al.; 2025; Liang et al.; 2026).

This approach stands out from the rest because of how highly adaptable it is. MAB-based algorithms can easily and confidently update their decisions as new information and performance feedback arrive. This is why they are a standout candidate for dynamic workloads.

A detailed taxonomy and feature comparison summarizing the literature alongside the characteristics of our proposed contextual MAB hyper-heuristic framework is presented in Table 1.

Table 1: Comprehensive Comparison of Literature and Features of the Proposed Work

Study / Work	Methodology / Core Strategy	Optimization Objectives	Target Environment	Dynamic Adaptivity	Context / Feature Awareness	Evaluation Engine
List-Based & Classical Heuristics						
Hosseini Shirvani and Noorian Talouki (2021)	HH-LiSch (Task duplication & slot insertion)	Makespan	Cloud VMs	Static / Low	×	Simulation
Adil et al. (2022)	CA-MLBS (SVM + PSO content classification)	Makespan, Throughput	Cloud	Static	✓ (Content Type)	CloudSim
Meta-Heuristic & Hybrid Approaches						
Kumar and Rai (2021)	Ant Colony Optimization (ACO)	Execution Time, Failure Rate	Cloud	Moderate	×	CloudSim
Alkaam et al. (2025)	HGHHC (Henry Gas + Harris Hawks + COBL)	Makespan	Heterogeneous Cloud	Moderate	×	HPC2N / NASA Traces
Bal et al. (2022)	RATS-HM (Cat Swarm + GO-DNN + Encryption)	Utilization, Power, Security, Latency	Cloud / Edge	Moderate	×	CloudSim
Machine Learning & Neural Network Methods						
Daid et al. (2021)	Probabilistic Neural Networks + Linear Regression	CPU Power, Allocation	Cloud Datacenters	High	✓ (CPU Load)	MATLAB (10 VMs)
Sravan and Shaik (2026)	Multi-Objective DRL Framework	Latency, Energy, SLA	Cloud / Fog	High	✓ (SLA Bounds)	Simulation
Multi-Armed Bandit (MAB) & Hyper-Heuristic Schedulers						
Singh et al. (2024)	Game-Theoretic MAB	Utilization, Latency, Fairness	Dynamic Cloud	High	×	Cloud Simulator
De Carvalho and Sichman (2025)	HHCSP (UCB-MAB + FRRMAB Credit Assignment)	Makespan, Cost, Carbon	Cloud (AWS / Teads)	High	×	Real AWS Data
Chigu et al. (2024)	UCB-based MAB Scheduler	Execution Latency	Edge-Cloud FaaS	High	×	AWS Lambda / OpenWhisk
Proposed Work	**Contextual MAB Hyper-Heuristic (UCB / Discounted-UCB / Thompson) + IPC Bridge**	**Makespan, Utilization, Non-stationary Regret**	**Heterogeneous Edge-Cloud**	**High (Online Adaptive)**	**✓ (Task Graph & Workload Features)**	**CloudSim Plus (Java) + Python IPC Bridge**

3 Methodology

3.1 Problem Formalization and System Model

Task scheduling in the cloud is seen as a problem where tasks arrive at the data center and need to be assigned to a virtual machine. Each virtual machine has a processing

capacity which is in Million Instructions per Second (MIPS), an operational cost rate and a baseline power draw. Each task has a size or length in Million Instructions, and a memory requirement.

The tasks are executed in batches. For each batch, a number of tasks arrives, and the scheduler selects a low level heuristic from a predefined pool of algorithms. The selected algorithm then determines the virtual machine each task is assigned to, which is then executed within CloudSim. Once the execution is complete, CloudSim returns netrics which the then transformed into a singular reward within the python program, to update the bandit's Q-values.

3.2 Low-Level Heuristic Pool

The low-level heuristic pool consists of nine scheduling algorithms:

1. **HEFT (Heterogeneous Earliest Finish Time):** Prioritizes tasks using upward rank values and assigning tasks to VMs with the earliest finish time.
2. **PEFT (Predicting Earliest Finish Time):** Extends HEFT by incorporating an Optimistic Cost Table (OCT) to evaluate the downstream path impact.
3. **Min-Min:** Assigns small tasks first to resources that execute them the fastest.
4. **Max-Min:** Prioritizes large tasks first to prevent longer duration jobs from monopolizing compute resources toward the end of execution.
5. **Round-Robin:** This is the simplest form of task scheduling. It assigns a task to the next available VM.
6. **Sufferage:** Assigns tasks based on sufferage value (the difference between a task's second-earliest and earliest finish times), prioritizing tasks that face the greatest penalty if delayed.
7. **HEFT with Duplication (HEFT-TD):** Duplicates critical predecessor tasks on redundant VM idle slots to eliminate inter-task communication latency.
8. **Simulated Annealing (SA):** A probabilistic local search meta-heuristic that escapes local optima by accepting inferior schedule moves according to a decaying temperature schedule.
9. **HGHHC:** A novel meta-heuristic integrating Henry Gas Solubility Optimization and Harris Hawks Optimization with Comprehensive Opposition-Based Learning (Alkaam et al.; 2025).

3.3 Multi-Armed Bandit Decision Algorithms

We evaluate four MAB selection mechanisms to manage the exploration-exploitation trade-off under dynamic cloud conditions:

3.3.1 ϵ-Greedy

Selects the arm with the highest estimated mean reward with probability $1 - \epsilon$, and explores a random arm with probability ϵ (Tokic; 2010). As shown in Equation (1):

$$a_k = \begin{cases} \arg\max_{a \in \mathcal{A}} \hat{Q}_k(a), & \text{with probability } 1 - \epsilon \\ \text{random arm } a \in \mathcal{A}, & \text{with probability } \epsilon \end{cases} \tag{1}$$

where $\epsilon = 0.1$, and $\hat{Q}_k(a)$ is the empirical mean reward of arm a up to round k.

3.3.2 Standard UCB

Constructs upper confidence bounds based on Hoeffding's Inequality to balance empirical performance and selection uncertainty (Shrilekhya et al.; 2025). Shown in Equation (2):

$$a_k = \arg\max_{a \in \mathcal{A}} \left[\hat{Q}_k(a) + c \cdot \sqrt{\frac{\ln k}{N_k(a)}} \right] \tag{2}$$

where $N_k(a)$ is the total pull count of arm a up to round k, and $c = \sqrt{2}$ is the exploration constant.

3.3.3 Discounted UCB (D-UCB)

Introduces a discount factor $\gamma \in (0, 1)$ to exponentially decay historical observations (De Carvalho and Sichman; 2025; Singh et al.; 2024) shown in Equations (3) and (4):

$$\hat{Q}_k(\gamma, a) = \frac{\sum_{i=1}^{k-1} \gamma^{k-1-i} R_i \mathbb{I}(a_i = a)}{N_k(\gamma, a)} \tag{3}$$

$$N_k(\gamma, a) = \sum_{i=1}^{k-1} \gamma^{k-1-i} \mathbb{I}(a_i = a) \tag{4}$$

The decision rule selects the arm maximizing the discounted upper bound shown in Equation (5):

$$a_k = \arg\max_{a \in \mathcal{A}} \left[\hat{Q}_k(\gamma, a) + 2B \sqrt{\frac{\xi \ln n_k(\gamma)}{N_k(\gamma, a)}} \right] \tag{5}$$

where $n_k(\gamma) = \sum_{a \in \mathcal{A}} N_k(\gamma, a)$, $\gamma = 0.95$, $B = 1.0$ represents the reward bound, and $\xi = 0.5$ tunes exploration intensity.

3.3.4 Scaled Thompson Sampling

Maintains continuous probability distributions over expected rewards. At round k, sample values are drawn for each arm $a \in \mathcal{A}$, and the arm with the maximum sample is selected (Shang et al.; 2024). Shown in Equation (6):

$$a_k = \arg\max_{a \in \mathcal{A}} \theta_a \tag{6}$$

3.4 Evaluation Methodology

To ensure rigorous empirical evaluation (Zhao et al.; 2025), our experimental methodology follows a structured pipeline:

3.4.1 Trace Datasets

We evaluate the system using two real-world high-performance computing workload traces from the Parallel Workloads Archive:

- **HPC2N Trace:** Workload trace containing long execution profiles and heavy resource demands. It can be found here.
- **NASA Ames iPSC/860 Trace:** Shorter duration scale trace with dynamic arrival bursts and high arrival density. It can be found here.

3.4.2 Non-Stationary Failure Injection

To confirm how adaptable the scheduler is, we simulate an infrastructure failure event, where we reduce the available computing resources by 90%.

4 Design Specification

This section outlines the architectural framework, functional requirements and algorithmic design of the system.

4.1 System Requirements and Architectural Constraints

To ensure that the scheduler can properly schedule tasks without any bottlenecks, as well as being adaptive, there are four key technical requirements that must be met:

1. **Decoupled Execution Architecture:** The control plane and the cloud simulation environment must run as standalone processes to ensure modular updates and avoid tight coupling.
2. **Low-Latency Inter-Process Communication (IPC):** The Java CloudSim environment is invoked within the Python program to ensure that the two processes can communicate with each other with low latency.
3. **Non-Stationary Adaptability:** The Python environment, which houses the scheduling algorithms must periodically update its states, in order to recover from changes in the cloud environment.
4. **Multi-Objective Metric Extraction:** At the end of each round, metrics such as makespan and carbon utilization are logged.

4.2 System Architecture and Streaming Pipe Pipeline

The system is a decoupled, event-driven system that is split into a Python decision engine and a Java CloudSim environment. Figure 1 provides a visualization of how everything comes together.

The interaction workflow proceeds as follows:

1. **Batch Arrival and Workload Characterization:** The Python program receives a batch of tasks from the workload trace file.

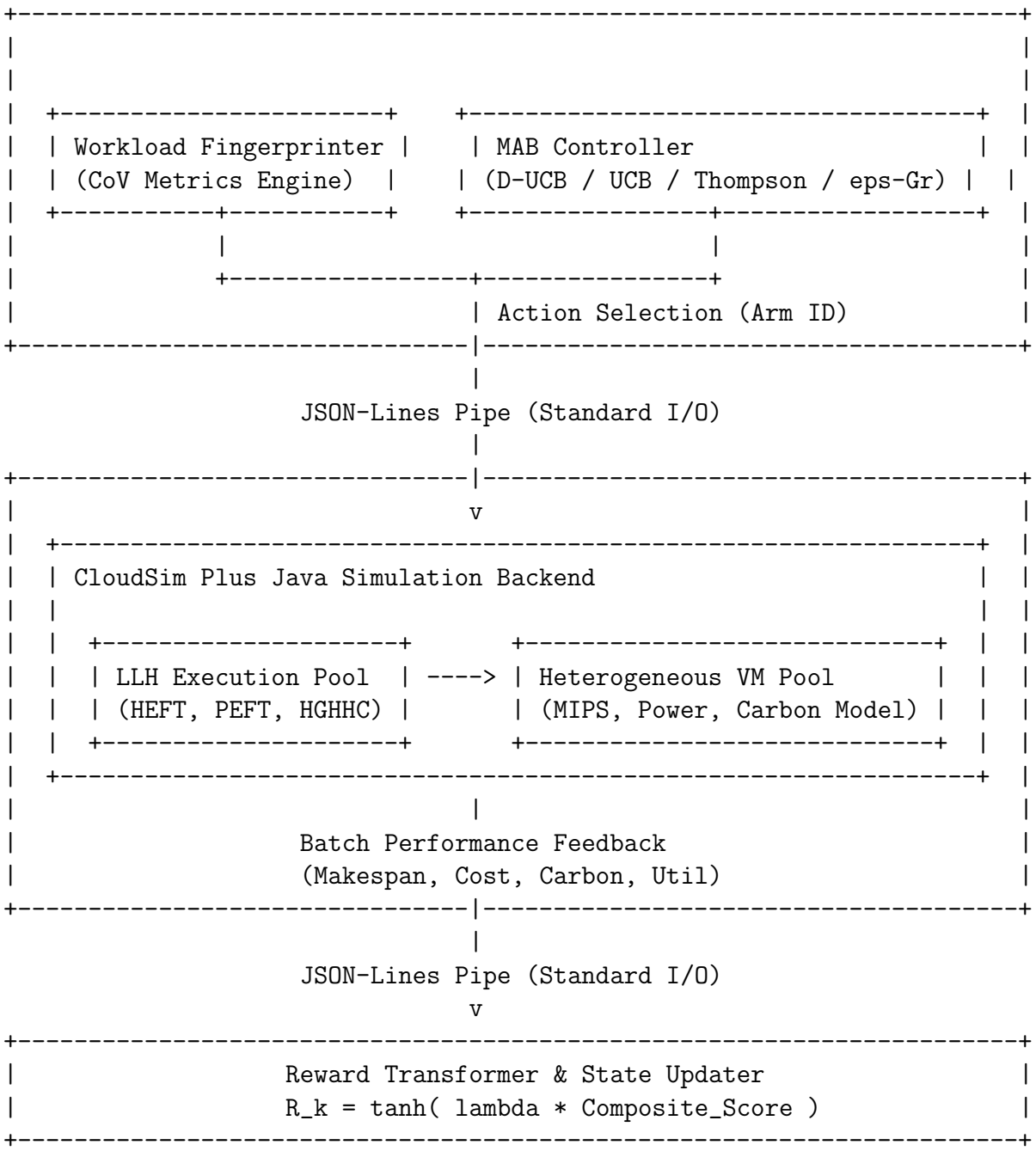


Figure 1: System Architecture showing the Python Hyper-Heuristic Engine linked via Standard I/O streaming pipes to the CloudSim Plus backend.

2. **Action Selection:** The MAB controller passes the tasks to its selection rule (ϵ-Greedy, UCB, Discounted UCB, or Thompson Sampling) to pick an arm.

3. **Command Streaming:** The engine packages arm and batch metadata into a JSON payload and writes it to standard output.

4. **Simulation Execution:** The Java CloudSim process reads standard input, deserializes the JSON object, instantiates the chosen low-level heuristic, schedules the tasks across the active VM pool, and runs the simulation step.

5. **Feedback Streaming:** Once the batch completes, CloudSim calculates makespan, financial cost, carbon emissions, and VM utilization, then serializes these metrics into JSON, and writes them back to standard output.

6. **Reward Squashing:** The Python program receives the feedback from the CloudSim environment, computes the reward within the range of -1 and 1, and updates the bandit's Q-values.

4.3 Algorithmic Functionality of the Hyper-Heuristic Model

Our core algorithm operates as a high-level policy wrapper over the pool of nine low-level heuristics.

4.3.1 Operational Logic

Algorithm 1 details the step-by-step decision framework running inside the Python orchestration controller.

Algorithm 1 Dynamic Workload-Aware Discounted MAB Hyper-Heuristic

Require: Arm space $\mathcal{A} = \{1, \ldots, 9\}$, discount factor $\gamma = 0.95$, exploration scale c_0, objective weights $\boldsymbol{w}$, total batches K.

Ensure: Schedule mappings and updated bandit parameters.

Initialize $N(a) \leftarrow 0$, $\hat{Q}(a) \leftarrow 0$ for all $a \in \mathcal{A}$.
for each batch decision round $k = 1$ to K **do**
  Read arriving task batch B_k from workload trace stream.
  Compute task length mean $\mu(B_k)$ and standard deviation $\sigma(B_k)$.
  Calculate trace volatility: $\mathrm{CoV}_k \leftarrow \sigma(B_k)/\mu(B_k)$.
  Auto-tune exploration scale: $c_k \leftarrow c_0 \cdot (1 + \tanh(\mathrm{CoV}_k - \mathrm{CoV}_{\text{baseline}}))$.
  if $k \leq |\mathcal{A}|$ **then**
    Select $a_k \leftarrow k$ {Warm-up phase: pull every arm once}
  else
    for each arm $a \in \mathcal{A}$ **do**
      Compute discounted pull count: $N_k(\gamma, a) \leftarrow \sum_{i=1}^{k-1} \gamma^{k-1-i} \mathbb{I}(a_i = a)$.
      Compute discounted reward sum: $\hat{Q}_k(\gamma, a) \leftarrow \frac{\sum_{i=1}^{k-1} \gamma^{k-1-i} R_i \mathbb{I}(a_i = a)}{N_k(\gamma, a)}$.
    end for
    Select arm: $a_k \leftarrow \arg\max_{a \in \mathcal{A}} \left[\hat{Q}_k(\gamma, a) + 2B \sqrt{\frac{\xi \ln \sum_b N_k(\gamma, b)}{N_k(\gamma, a)}} \right]$.
  end if
  Package $\{a_k, B_k\}$ into JSON payload and emit over standard I/O pipe.
  Read execution feedback $\{M_k, C_k, E_k, U_k\}$ from CloudSim Plus pipe.
  Compute relative efficiency metrics $\Delta M_k, \Delta C_k, \Delta E_k$ against Round-Robin baseline.
  Compute raw multi-objective score: $S_k \leftarrow w_1 \Delta M_k + w_2 \Delta C_k + w_3 \Delta E_k + w_4 U_k$.
  Squash reward: $R_k \leftarrow \tanh(\lambda \cdot S_k)$.
  Update Q-values and historical observation logs with decay factor γ.
end for

5 Implementation

This section describes the final implementation of the dynamic Multi-Armed Bandit (MAB) hyper-heuristic system. It explains the software stack, the interprocess communication, low-level heuristic pool and outputs.

5.1 Technology Stack and Software Architecture

- **Python Execution Engine (v3.11.7):** This is the entry point to the scheduler that houses the multi-armed bandit decision algorithms, workload trace ingestion module, and the reward function. It uses `NumPy` and `SciPy` for statistical operations and `Pandas` for trace parsing.
- **Java Simulation Engine (OpenJDK 21 / CloudSim Plus v7.3.0):** Acts as the cloud simulation environment. CloudSim Plus core classes were extended to support low-level heuristic instantiation and I/O streams

5.2 Standard I/O Inter-Process Communication Pipeline

In order to reduce the overhead of the spawning a new process for every CloudSim instance, we make use of inter-process communication to create a singular data stream between the Python control plane and the Java simulation engine.

The Python program launches the Java instance as a managed subprocess that receives a JSON payload containing the batch index and task specifications.

The Java application holds a stream reader that parses incoming JSON records asynchronously, which then executes the batch across the virtual machine pool.

5.3 Low-Level Heuristic Pool Implementation

The low-level heuristic pool is implemented inside the Java simulation core under a unified strategy interface, with nine distinct scheduling routines being implemented:

1. **List-Based Schedulers (HEFT, PEFT, Min-Min, Max-Min, Sufferage, HEFT-TD):** Custom task prioritization routines construct directed acyclic graph (DAG) task ranks and evaluate expected finish times across heterogeneous VM capacity matrices on the fly.

2. **Deterministic Baseline (Round-Robin):** Sequentially maps incoming cloudlets to active VMs to establish a dynamic baseline for performance normalization.

3. **Local Search and Meta-Heuristics (Simulated Annealing, HGHHC):** Algorithmic state models execute local schedule permutations HGHHC combines the Henry Gas Solubility Optimization and the Harris-Hawks Optimization methodologies into a singular algorithm that runs multiple evaluations within given time slots.

5.4 Energy, Carbon Footprint, and Cost Modeling

In order to accurately measure the trade-offs, some CloudSim classes had to be modified, as well as creating new modules:

- **Monetary Cost Model:** With the help of AWS billing API, we were able to simulate resource pricing. Each VM instance incurs a fixed per-second compute rate based on its allocated processing capacity (MIPS) and memory profile, following tier-based provider pricing schemes.

- **Dynamic Power Model:** Power consumption $P(u)$ is calculated as a function of CPU utilization:

$$P(u) = P_{\text{idle}} + (P_{\text{max}} - P_{\text{idle}}) \times u \tag{7}$$

  where P_{idle} represents the VM's power consumption while idle, and P_{max} represents peak power under full load.

5.5 Non-Stationary Infrastructure Fault Injection

A fault injection module was implemented in the Python environment. The purpose of this module is to simulate infrastructure failure, as well as monitoring and evaluation how each bandit algorithm withstands a sudden change in VM capacity.

5.6 Generated Outputs and Data Transformation

The execution pipeline produces outputs for anaylysis:

1. **Raw Trace Log Stream:** Logs per batch that contain details such as the heuristic used, makespan, reward, etc.

2. **Aggregated Seed Benchmark Dataset:** CSV files for each arm under each of the bandit algorithms.

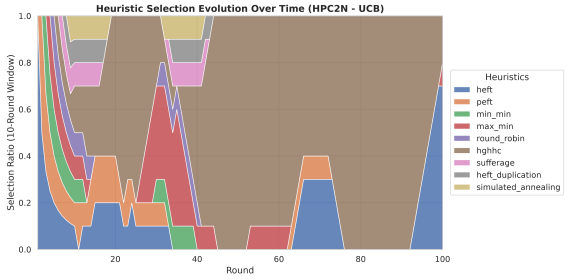
6 Evaluation

This section covers the analysis and auditing of the scheduler evaluated on real-world workload traces.

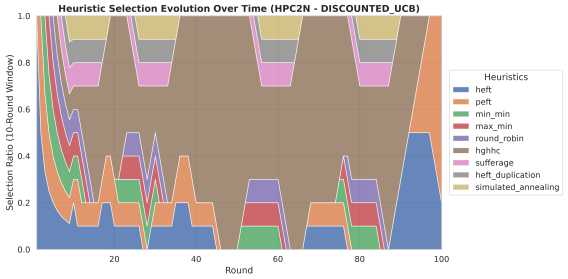
6.1 Experimental Results and Comparative Analysis

6.1.1 Heuristic Selection Ratio Evolution

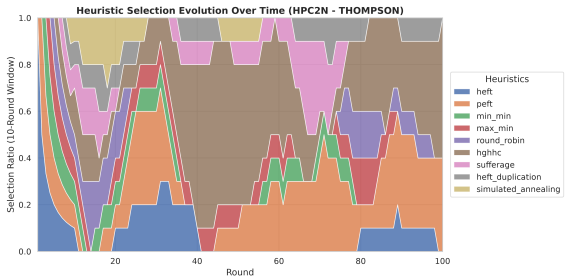
Figures 2 and 3 track how the selection of the low-level heuristic evolves over 100 simulation rounds, using a 10-round window.



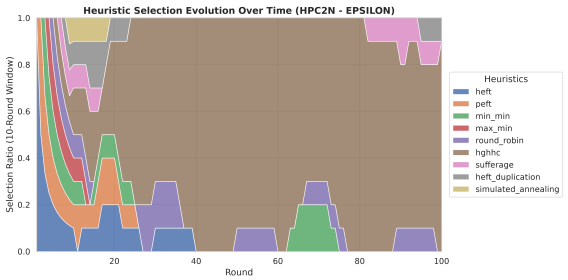
(a) HPC2N – UCB



(b) HPC2N – Discounted UCB



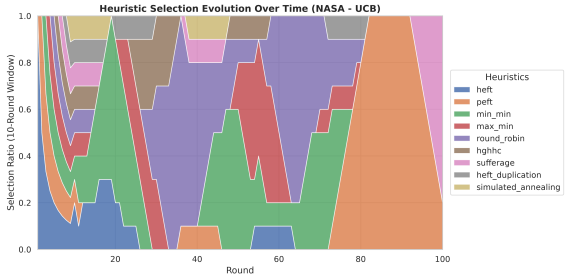
(c) HPC2N – Thompson Sampling



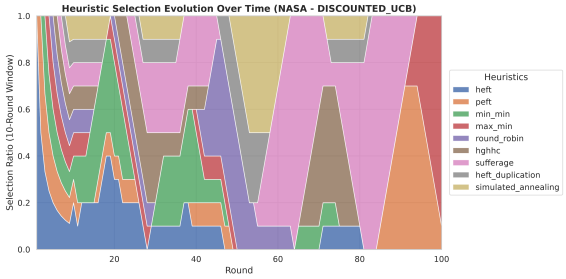
(d) HPC2N – ϵ-Greedy

Figure 2: Low-level heuristic selection ratio evolution over 100 rounds on the HPC2N trace.

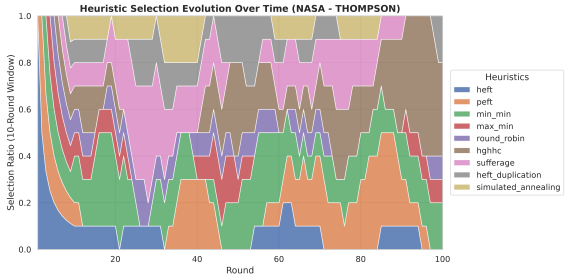
- **Initial Exploration Phase ($k \leq 10$):** Unbiased initial sampling shows a fair, random distribution over the selected algorithms



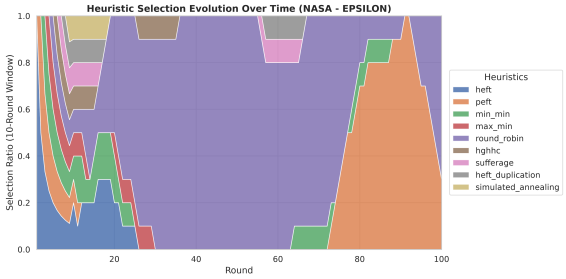
(a) NASA – UCB



(b) NASA – Discounted UCB



(c) NASA – Thompson Sampling



(d) NASA – ϵ-Greedy

Figure 3: Low-level heuristic selection ratio evolution over 100 rounds on the NASA Ames trace.

- **Exploitation Regime on HPC2N ($k > 15$):** On the HPC2N trace, UCB (Figure 2a) and Discounted UCB (Figure 2b) decide to focus on `hghhc`, which occupies up to 80% of selection windows between rounds 40 and 90, while also triggering `heft` and `peft` during shifts around round 65 and 95. Thompson Sampling (Figure 2c) maintains a more stable selection between `hghhc`, `peft`, and `sufferage`.

- **Phase Lock on NASA Ames:** On the NASA Ames trace, UCB (Figure 3a) aggressively chooses `round_robin` and `max_min` between rounds 30 and 60 before moving on to `peft`. Discounted UCB (Figure 3b) constantly goes back and forth between `sufferage`, `simulated_annealing`, `hghhc`, and `peft`.

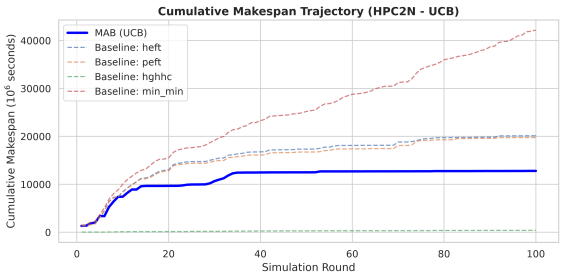
6.1.2 Cumulative Makespan Trajectories

Figures 4 and 5 showcase the cumulative makespan curves across all four bandit implementations against baseline heuristics.

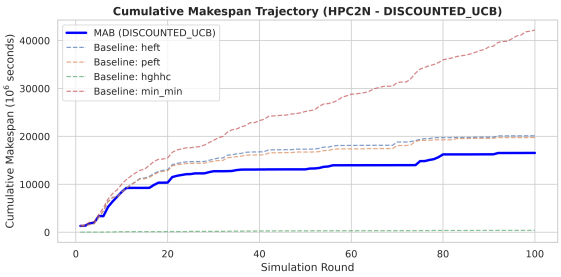
The primary empirical insights include:

- **Mitigation of Baseline Degradation:** Static `min_min` exhibits severe performance degradation on NASA Ames (Figures 5a–5d), accumulating over 1.6×10^9 seconds in makespan by round 100 due to severe task queueing. All MAB controllers avoid this failure mode by moving away from `min_min` as early into the batch as possible.

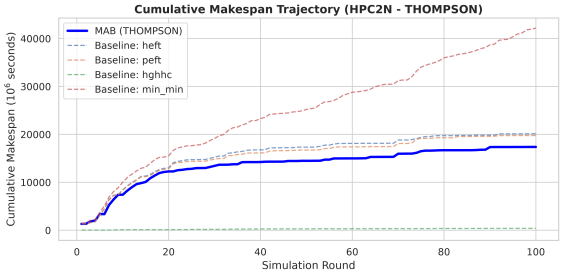
- **Performance Superiority on HPC2N:** On HPC2N (Figure 4), standard UCB, Discounted UCB, Thompson Sampling, and ϵ-Greedy all show a 59.5%–71.4% improvement over `min_min` and constantly outperforms the static list schedulers.



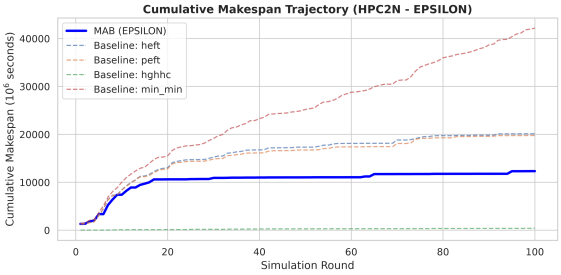
(a) HPC2N – UCB



(b) HPC2N – Discounted UCB

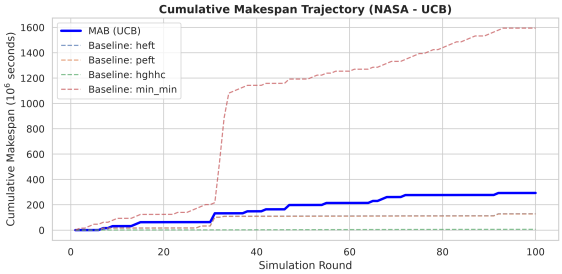


(c) HPC2N – Thompson Sampling

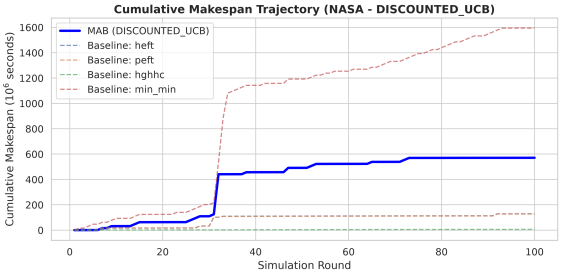


(d) HPC2N – ϵ-Greedy

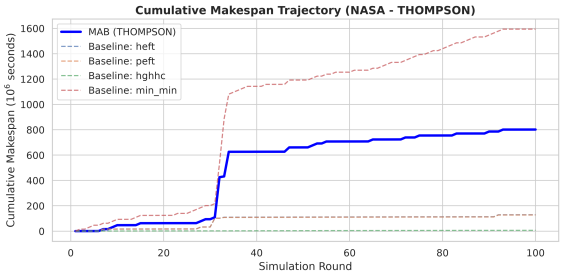
Figure 4: Cumulative makespan trajectories across MAB variants on the HPC2N workload.



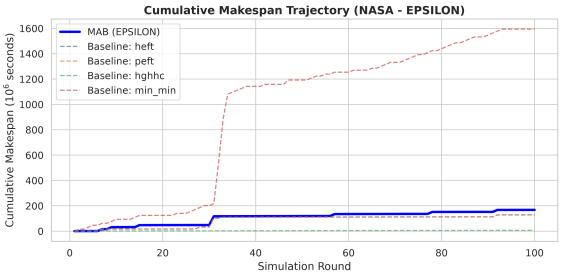
(a) NASA – UCB



(b) NASA – Discounted UCB



(c) NASA – Thompson Sampling



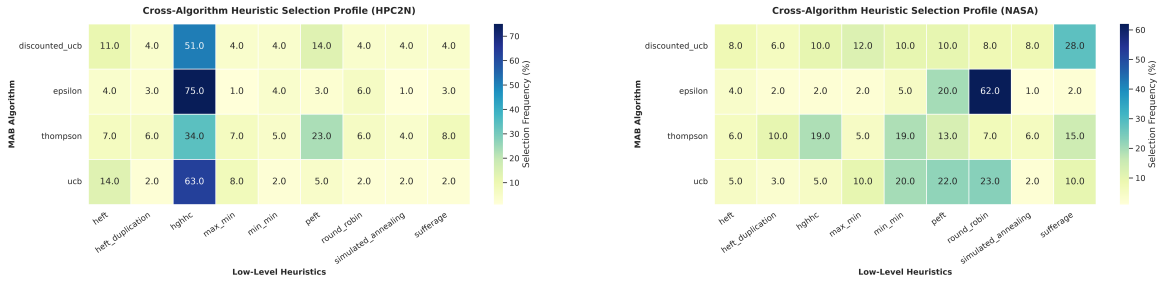
(d) NASA – ϵ-Greedy

Figure 5: Cumulative makespan trajectories across MAB variants on the NASA Ames workload.

- **Exploration Penalty under Thompson Sampling:** On NASA Ames (Figure 5c), Thompson Sampling accumulates 8.0×10^8 seconds makespan, whereas ϵ-Greedy (Figure 5d) terminates at 1.7×10^8 seconds, which shows that posterior sampling can lead to unnecessary exploration overhead in fairly predictable workloads.

6.1.3 Cross-Algorithm Selection Profile Heatmaps

Figures 6a and 6b show how frequently each low-level heuristic was selected across all four MAB controllers on both the HPC2N and NASA workload traces.



(a) HPC2N Trace Selection Profile

(b) NASA Ames Trace Selection Profile

Figure 6: Cross-algorithm low-level heuristic selection frequency profiles (%) across MAB variants for the HPC2N and NASA Ames workload traces.

Key selection tendencies across workload domains include:

1. **Primary Arm Preference (HPC2N):** On the HPC2N trace (Figure 6a), `hghhc` dominates selection across all MAB algoorithms due to the fact that it can efficiently schedule the tasks into available virtual machines.

2. **Workload-Driven Policy Shift (NASA Ames):** On the NASA Ames trace (Figure 6b), the selection density shifts dramatically away from pure metaheuristics toward list-based and priority-driven heuristics. `peft` and `sufferage` emerge as preferred arms under UCB and Discounted UCB (accounting for $\approx$ 38.0% and 26.0% of total selections, respectively), while `round_robin` and `max_min` are sparsely engaged during high-concurrency rounds.

3. **Filtering Inefficient Arms:** Inefficient heuristics are discarded, accounting for $\leq 4.0\%$ of total choices across all MAB algorithms.

6.2 Critical Appraisal and Methodological Limitations

We identify three methodological limitations of this study:

6.2.1 Hyperparameter Tuning and Exploration Bounds

The exploration factor $c = \sqrt{2}$ in UCB and discount factor $\gamma = 0.95$ in Discounted UCB were set as a constant, which can lead to longer exploration phases than needed. Dynamically setting these values will make sure that the scheduler finds a balance between exploration and exploitation.

6.2.2 Simulation Idealization vs. Real-World Cloud Dynamics

For the purpose of this project, we make use of CloudSim, which executes tasks predictably. But in real-world environments, this will never be the case as there will be network jitter, VM cold starts and so on, which will introduce variance into execution times.

To bridge this gap, we can make three extensions:

- **Asynchronous Policy Update:** Instead of periodic execution, using an asynchronous architecture that updates metrics instantly
- **Adaptive Reward Smoothing:** Using a moving average to detect anomalies and prevent them from affecting historical rewards.
- **Fallback and Safety Bounds:** Hard caps that force a baseline heuristic to be selected when exploitation is taking to long.

6.3 Suggested Modifications and Design Improvements

To address these limitations, we propose three enhancements:

1. **Adaptive Memory Decay (γ_k):** We can dynamically adjust the discount factor for the older scheduled tasks based on how volatile the incoming workload is.
2. **Contextual Bandits:** Incorporate state vectors to enable state-aware heuristic selection.
3. **Testbed Deployment:** Testing the performance scheduler on a Kubernetes cluster that mimics real-world scenarios.

7 Conclusion and Future Work

In this study, we tried to answer the question of whether an adaptive Multi-Armed Bandit hyper-heuristic framework could consistently schedule tasks in a heterogeneous cloud environment in real time, while keeping track of multiple objectives such as makespan and energy consumption. The process of getting the answer involved developing a scheduling architecture that incorporates a range of scheduling algorithms, from list-based schedulers to hybrid algorithms. A reward metric that is the output of a function that computes the makespan, cost and carbon footprint of the simulated machines run within CloudSIm showed that the framework is viable.

The value of this work lies in establishing the Multi-Armed Bandit approach as a real and practical compromise between static list-based schedulers and more compute-heavy metaheuristics. This approach helps to eliminate the need for offline training, and it enables automated adaptability.

Future research should try and transition to a contextual Multi-Armed Bandit approach because it takes more into account before making a decision, as opposed to a context-free MAB that goes for the globally best option. Opting for a Deep Reinforcement Learning approach is also an avenue to explore, as the decisions made in a round carry delayed consequences in future rounds.

References

Adil, M., Nabi, S., Aleem, M., Diaz, V. G. and Lin, J. C. (2022). ¡scp¿ca-mlbs¡/scp¿: Content-aware machine learning based load balancing scheduler in the cloud environment, *Expert Systems* **40**(4).

Alkaam, N. O., Sultan, M., Hussin, M. and Sharif, K. Y. (2025). Hybrid henry gas-harris hawks comprehensive-opposition algorithm for task scheduling in cloud computing, *IEEE Access* pp. 1–1.

Arora, R., Devi, U., Eilam, T., Goyal, A., Narayanaswami, C. and Parida, P. (2023). Towards carbon footprint management in hybrid multicloud, *Proceedings of the 2nd workshop on sustainable computer systems* pp. 1–7.

Bal, P. K., Mohapatra, S. K., Das, T. K., Srinivasan, K. and Hu, Y.-C. (2022). A joint resource allocation, security with efficient task scheduling in cloud computing using hybrid machine learning techniques, *Sensors (Basel, Switzerland)* **22**(3).
URL: *https://eds.p.ebscohost.com/eds/pdfviewer/pdfviewer?vid=0&sid=2b2c3ba3-fb23-48d6-833f-6c4da057035f%40redis*

Chigu, J., El-Mahdy, A., Mokhtar, B. and Elsabrouty, M. (2024). A low-latency edge-cloud serverless computing framework with a multi-armed bandit scheduler, *2024 International Wireless Communications and Mobile Computing (IWCMC)* pp. 1655–1660.

Choppara, P. and Lokesh, B. (2025). Efficient task scheduling and load balancing in fog computing for crucial healthcare through deep reinforcement learning, *IEEE access* **13**: 26542–26563.

Daid, R., Kumar, Y., Hu, Y.-C. and Chen, W.-L. (2021). An effective scheduling in data centres for efficient cpu usage and service level agreement fulfilment using machine learning, *Connection Science* **33**(4): 954–974.

De Carvalho, V. R. and Sichman, J. S. (2025). Toward cost-efficiency and reduced carbon footprint: A multi-armed bandit hyper-heuristic for cloud scheduling problems, *IEEE Access* **13**: 204656–204675.

Hosseini Shirvani, M. and Noorian Talouki, R. (2021). A novel hybrid heuristic-based list scheduling algorithm in heterogeneous cloud computing environment for makespan optimization, *Parallel Computing* **108**: 102828.

Jeba, J. A., Roy, S., Rashid, M. O., Atik, S. T. and Whaiduzzaman, M. (2019). Towards green cloud computing an algorithmic approach for energy minimization in cloud data centers, *Int. J. Cloud Appl. Comput.* **9**(1): 59–81.
URL: *https://doi.org/10.4018/IJCAC.2019010105*

Jun, L., Haiyang, S., Xiumei, D., Kang, W., Long, S., Le, L. and Wen, C. (2025). Mobility-aware user scheduling in wireless federated learning with contextual multi-armed bandit, *China Communications* **22**(11): 256–272.

Khan, F., Shah, I., Jan, S., Ahmad, S. and Whangbo, T. (2025). Machine learning-based resource management in fog computing: A systematic literature review, *Sensors* **25**(3): 687.

Kumar, A. and Rai, D. (2021). Implementation of machine learning algorithm for task scheduling in cloud computing environment, *Turkish Online Journal of Qualitative Inquiry (TOJQI)* **12**(6): 5201–5209.

Kumar, M., Sharma, S., Goel, A. and Singh, S. (2019). A comprehensive survey for scheduling techniques in cloud computing, *Journal of Network and Computer Applications* **143**: 1–33.
URL: *https://www.sciencedirect.com/science/article/pii/S1084804519302036*

Liang, H., Liu, C., Song, Y., Yin, Z. and Wang, G. (2026). Joint two stage beamforming and user scheduling for leo satellite communications via hierarchical multi-armed bandit, *IEEE Transactions on Cognitive Communications and Networking* **12**: 1918–1932.

Mathew, T., Sekaran, K. C. and Jose, J. (2014). Study and analysis of various task scheduling algorithms in the cloud computing environment, *2014 International Conference on Advances in Computing, Communications and Informatics (ICACCI)* .

Saurav, S. K. and Benedict, S. (2021). A taxonomy and survey on energy-aware scientific workflows scheduling in large-scale heterogeneous architecture, *2021 6Th international conference on inventive computation technologies (icict)* pp. 820–826.

Shang, Z., Yang, L. and Li, M. (2024). Two-step thompson sampling for multi-armed bandit problem, *2024 43rd Chinese Control Conference (CCC)*, pp. 5802–5805.

Shrilekhya, N. S., Sai, N. K. and Seshadri, K. (2025). Upper confidence bound versus adaptive posterior sampling for multi-armed bandit game models, *2025 International Conference on Innovative Trends in Information Technology (ICITIIT)*, pp. 1–6.

Singh, S., Chakraborty, S., Gourisaria, M. K., Swain, S., Raj, R. and Bandyopadhyay, A. (2024). Leveraging multi-armed bandits for cloud resource optimization, *2024 First International Conference on Software, Systems and Information Technology (SSITCON)* pp. 1–6.

Sravan, P. and Shaik, M. A. (2026). Drl-based multi-objective task scheduling for edge-cloud computing: Latency, energy, and sla optimisation, *Scientific reports* **16**(1).

Tokic, M. (2010). Adaptive ϵ-greedy exploration in reinforcement learning based on value differences, *in* R. Dillmann, J. Beyerer, U. D. Hanebeck and T. Schultz (eds), *KI 2010: Advances in Artificial Intelligence*, Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 203–210.

Tran-Dang, H. and Kim, D.-S. (2024). Parallel computation in dynamic fog computing networks: A multi-armed bandit learning-based decentralized matching approach, *2024 IEEE International Conference on Joint Cloud Computing (JCC)* pp. 57–60.

Zhao, N., Wang, H., Yang, F., Zhang, J., Wu, R., Zhong, T., Han, S., Huang, Z., Zhang, X. and Zhao, X. (2025). Scheduling virtual machines and containers: A comparative review of techniques, performance, and future trends, *Journal of Systems Architecture* **168**: 103583–103583.